

Heart Rate Monitor Project

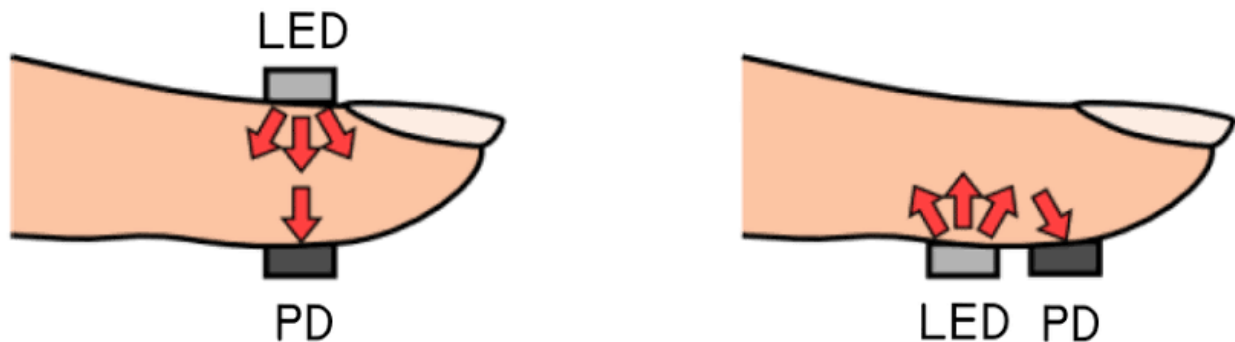
Goal:

- Build a low cost heart rate detector
- Measure somewhere around 40-240 BPM while rejecting noise

Requirements:

- Use a single 9v battery
- Output voltages safe enough for esp32 adc to read ($<3.3\text{ V}$) while still big enough to notice a difference (12 bit ADC over range from 0-3.3 V means around an 8mV step)

To measure the heartbeat I saw many implementations use an IR LED, so I decided to use that too. Luckily I had an IR LED and photodiode laying around. Of those implementations I saw some use a phototransistor, but the photodiode does the job of outputting current proportional to the intensity of the IR light emitted, so I used that. There are 2 ways that these 2 components can be set up, transmission or reflection. I chose transmission because this reduces motion artifacts and ambient noise (Later on I confirmed it gave more accurate readings).

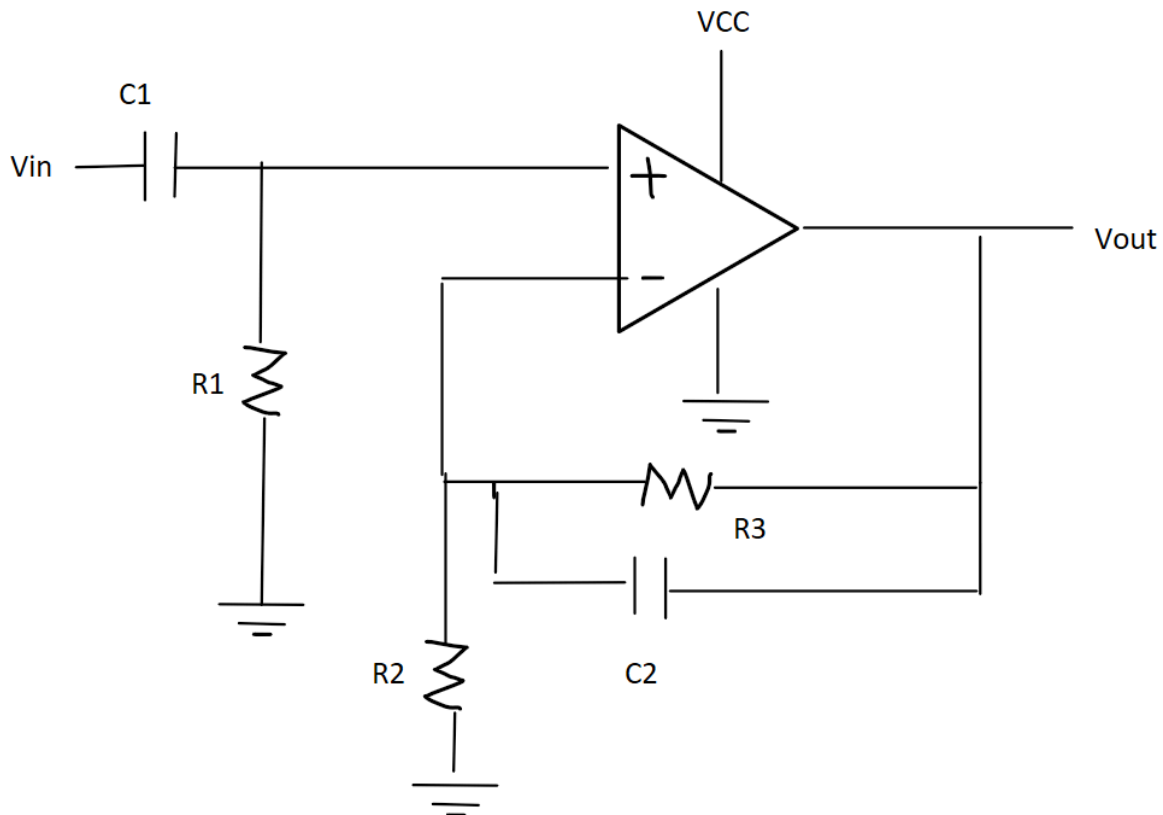


I set up the IR LED to output constantly by powering it with the 9v and the appropriate resistor in series. My finger would be placed between the IR LED and the photodiode. After some testing, I measured the current produced by the photodiode (no obstruction) to be 60-70 μA (which corresponded with the datasheet) and 1-3 μA with my finger obstructing.

To use this as an input to my opamp I used a resistor to convert the current into voltage. I chose a value of 100k. This would produce 6-7 V no obstruction and 100-300 mV (hundreds of millivolts). To analyze my circuit with the simulator I modeled the photodiode as an AC current source with an offset and chose $0.1 \cdot \sin(2\pi \cdot f \cdot t) + 5\text{ }\mu\text{A}$. The reasoning behind the AC magnitude of 0.1 was that I wasn't able to easily see the measured voltage swing so I assumed it was 1/10.

Transfer function calculation:

I knew that I wanted to reject too low and too high frequencies while also amplifying the input voltage so an active bandpass filter is what I needed. Online I found a version of the circuit that used a single opamp and some resistors and capacitors to achieve this.



I then derived the transfer function V_{out}/V_{in} :

Assuming ideal opamp, $V_p = V_n$ (non inverting and inverting input nodes of opamp)

$$1. \quad V_p = V_n = V_{in} * R1 / (R1 + 1/(sC1)) = sR1C1 / (sR1C1 + 1) * V_{in}$$

KCL at node V_n

$$2. \quad V_n / R2 = (-V_n + V_{out}) / (R3 \parallel 1/(sC2))$$

$$3. \quad (R3 \parallel 1/(sC2)) / R2 = R3/R2 * 1/(sR3C2 + 1) = V_{out}/V_n - 1$$

$$4. \quad R3/R2 * 1/(sR3C2 + 1) + 1 = V_{out}/V_{in} * (sR1C1 + 1)/(sR1C1)$$

$$5. \quad V_{out}/V_{in} = R3/R2 * 1/(sR3C2 + 1) * sR1C1 / (sR1C1 + 1) + sR1C1 / (sR1C1 + 1)$$

$$6. \quad V_{out}/V_{in} = sR1C1 / (sR1C1 + 1) * (R3/R2 * 1/(sR3C2 + 1) + 1)$$

$$7. \quad V_{out}/V_{in} = sR1C1 / (sR1C1 + 1) * ((R3/R2 + (sR3C2 + 1)) / (sR3C2 + 1))$$

The transfer function is composed of a highpass and lowpass, so it matches the bandpass behavior that I wanted.

Picking component values:

For the low cutoff frequency I picked 40 BPM, so:

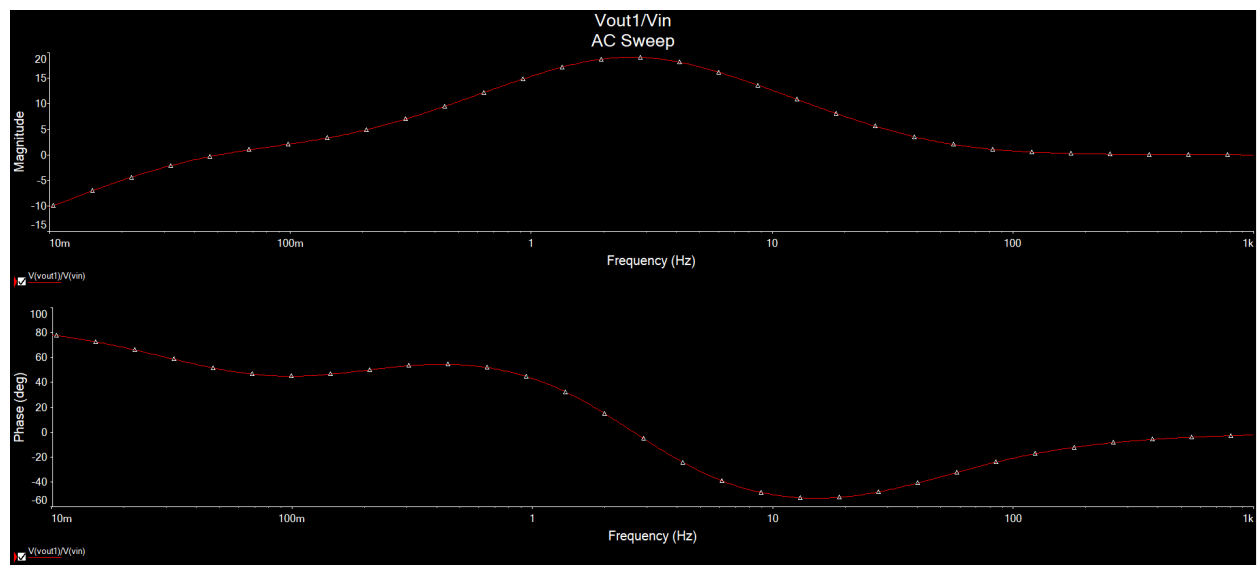
1. $1/(R1C1*2\pi) = 40/60$
2. $R1C1 = 0.2387$

For the high cutoff frequency I picked 240 BPM, so:

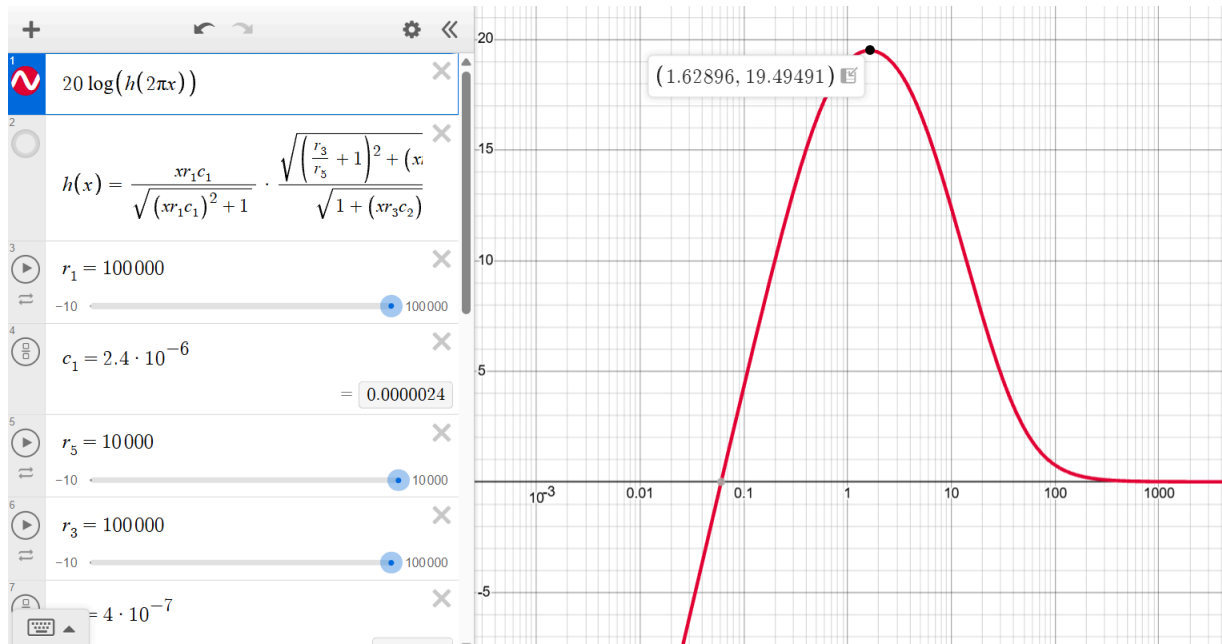
1. $1/(R3C2*2\pi) = 240/60$
2. $R3C2 = 0.0398$

I picked $R3 = 100k$ and $R2 = 10k$. This meant that $C2 = .4 \mu F$ and $C1 = 2.4 \mu F$

I then plotted the magnitude response plot on Multisim as well as Desmos to confirm the behavior.



Bode plot of first opamp output / input voltage



Magnitude response of first opamp on Desmos (y axis = dB, x axis = Hz)

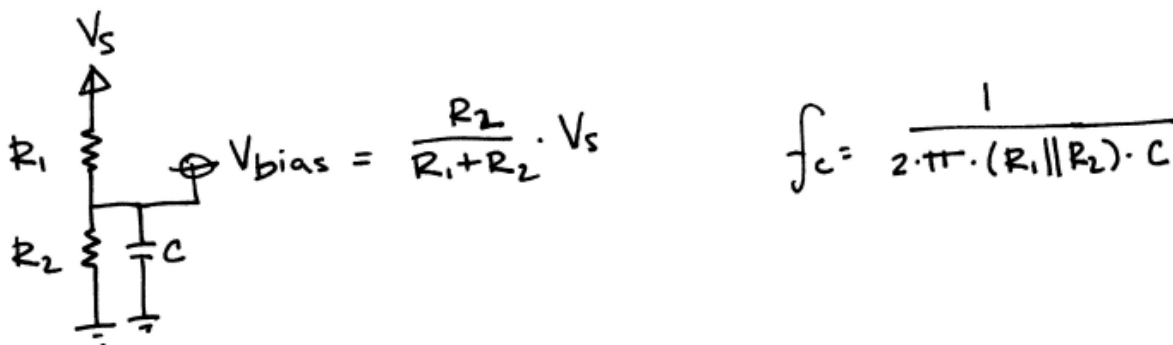
Opamp biasing

Since I planned on the output of the opamp to be directly read by the ESP32 ADC, I needed to make sure that the voltage was < 3.3 V the maximum readable voltage. This meant that I had to bias the output voltage to 1.65V to get the most possible output swing both up and down. Without biasing, the input AC signal which has no DC offset (so 0V) would be amplified but the negative voltages would be cut off in the output since I chose a single supply (0-9V).

I reviewed the following resource on biasing opamps:

https://fab.cba.mit.edu/classes/863.16/section.Architecture/people/Ge/bias_opamp.pdf

I needed to create 1.65V using the following circuit:



The value of f_c has to be low enough so not to affect the magnitude response of the actual circuit. The capacitor C helps with removing noise from the DC supply so V_{bias} is stable. The

values I chose initially for the resistors were $R1 = 4.5k$ and $R2 = 1k$ and $f_c = 0.1$, an amount below the 0.667 (40 BPM) low cutoff of the bandpass.

Calculating C:

1. $0.1 = 1 / (2\pi * (4.5k \parallel 1k) * C)$
2. $C = 1 / (0.1 * 2\pi * 818.18) = 0.00195$

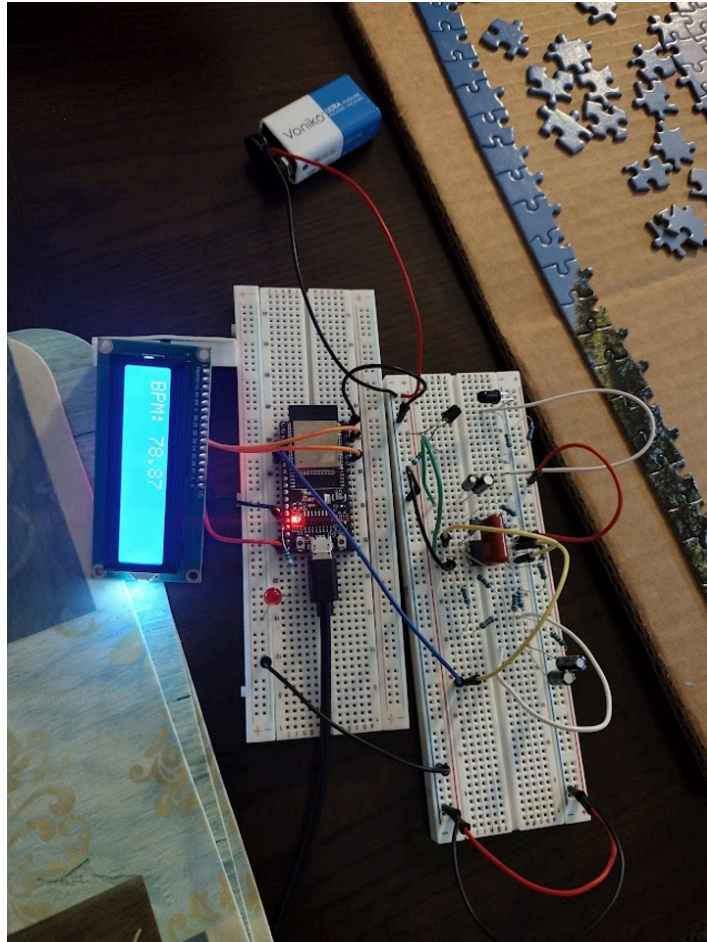
That is in the thousands of microfarads which is too big, so I increased both the resistors by a factor of 10. $R1 = 45k$ and $R2 = 10k$. That way $C = 194.52 \mu F$.

Fixing gain issue

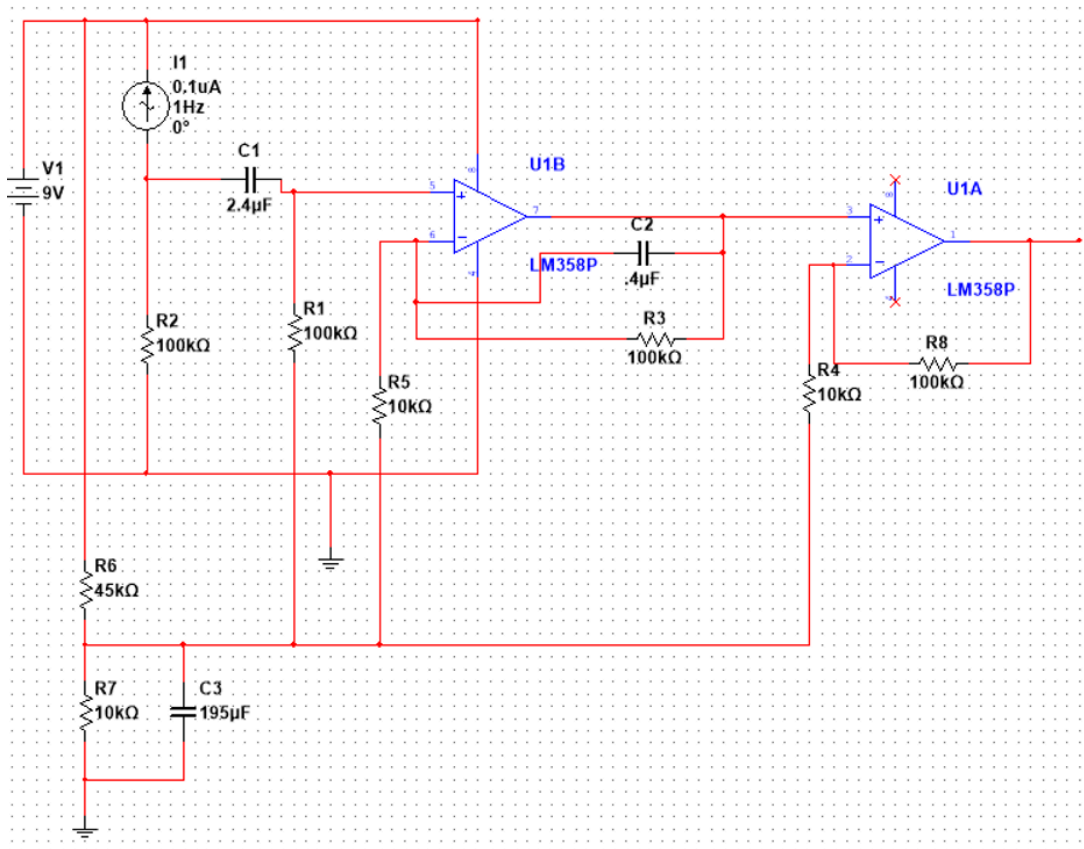
With the gain of 10, I was only seeing a swing at most of 0.1V at the output, when ideally I wanted 1V. I had the choice of either increasing the gain of the single first stage to 100 or adding another stage that had a gain of 10.

I tried the first choice, but whenever I obstructed the IR LED with my finger or any object, the DC bias would get closer to 0V instead of 1.65V, so the lower end of the waveform would get cut off easily. This made me go with the second choice, so I used the second opamp in the LM358 as a simple non inverting opamp.

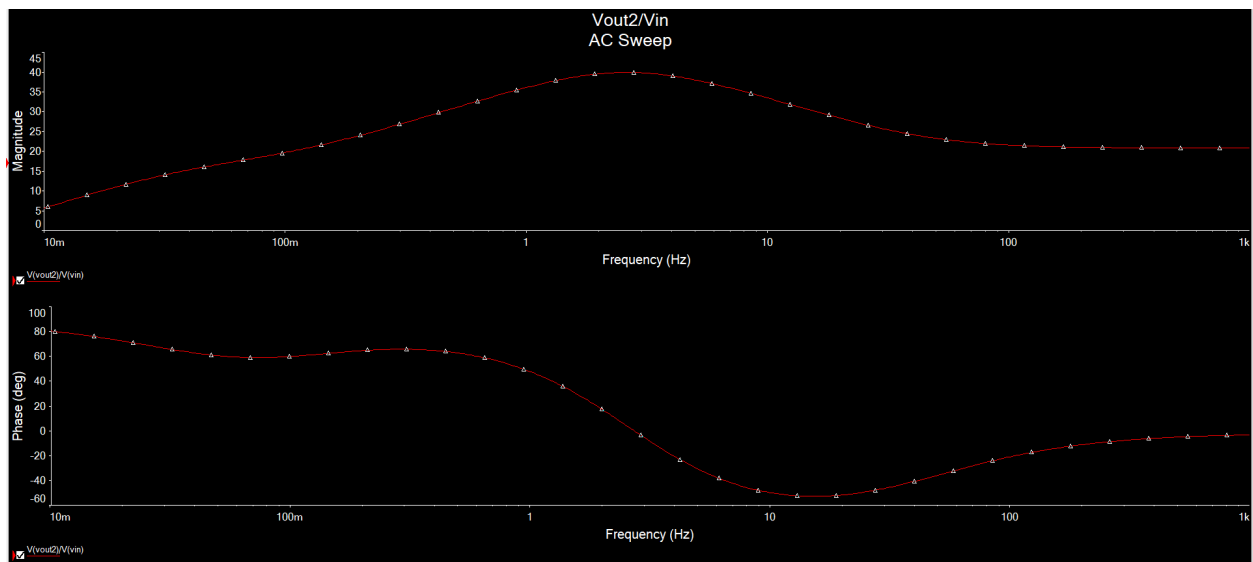
The downside of this approach is that this raises the magnitude response +20 db (x10) across the whole frequency range instead of keeping unwanted frequencies at 0db or less. The positive side is that the actual signal to noise ratio is still the same (ex. 0.01V noise 0.1V signal -> 0.1V noise 1V signal).



Circuit

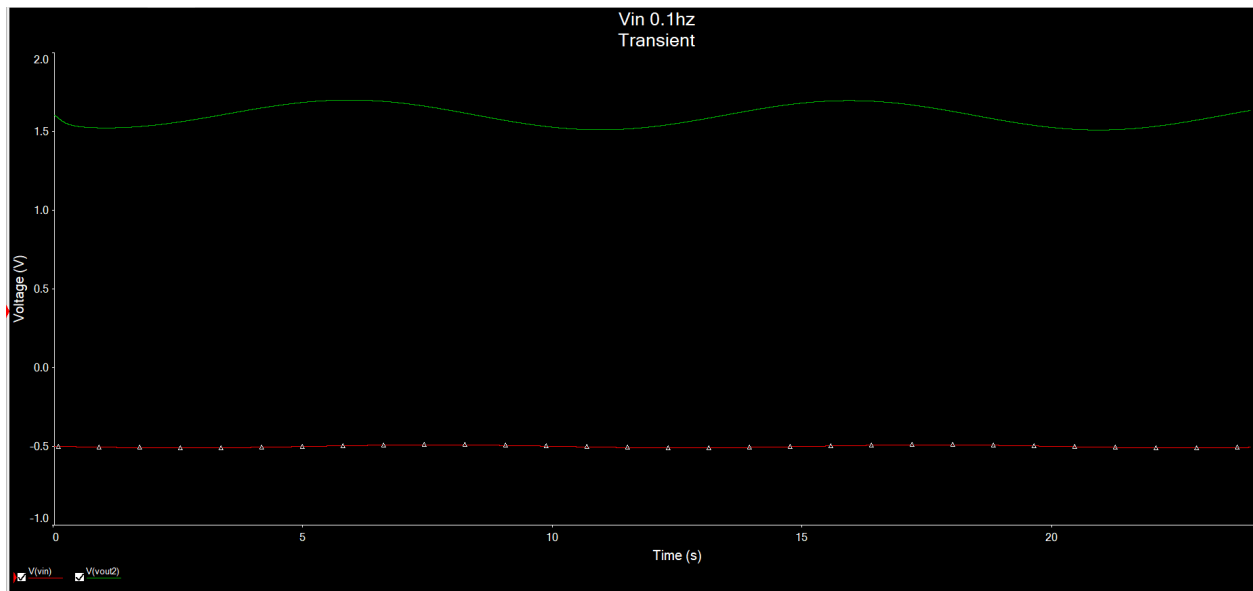


Multisim circuit diagram

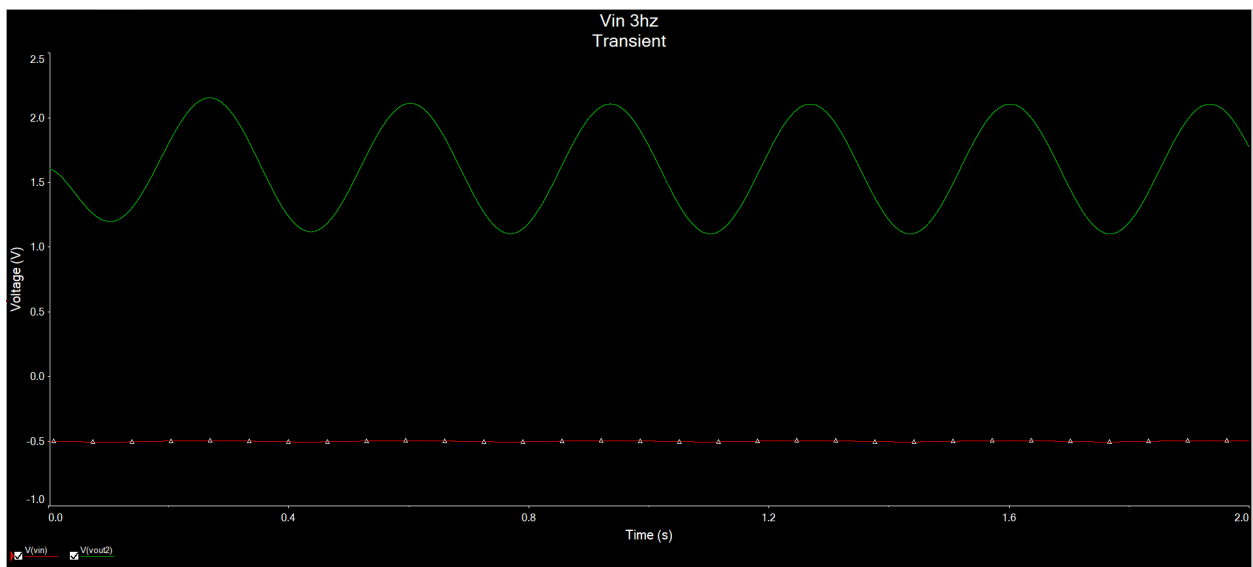


Bode plot of second opamp output / input voltage (+20 dB)

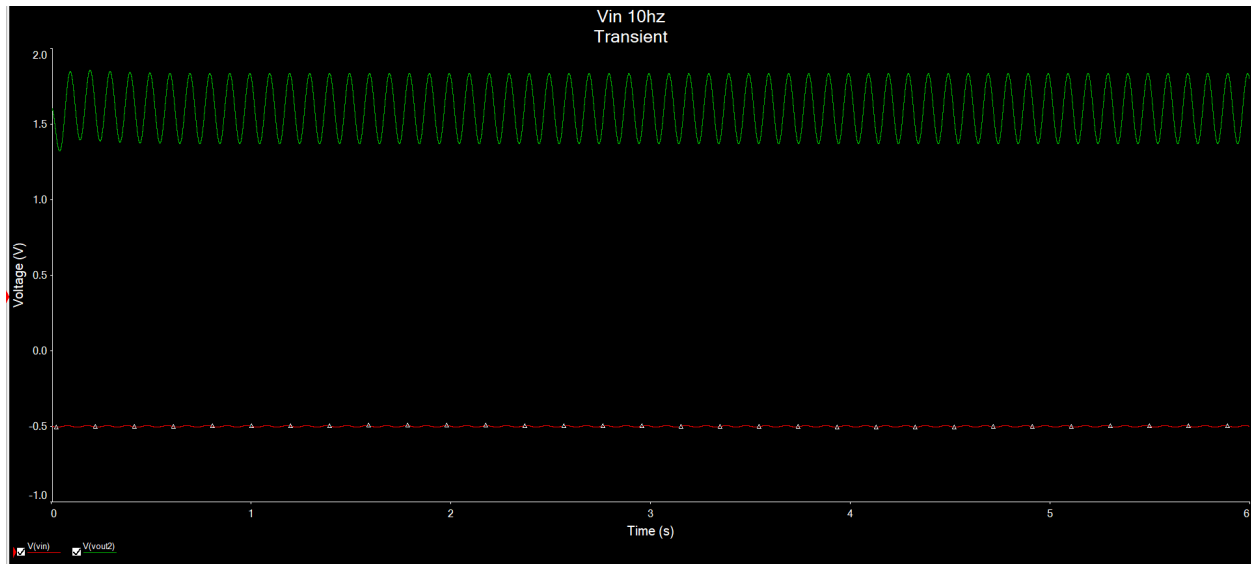
The following are transient graphs with the current source = $0.1 \cdot \sin(2\pi \cdot f \cdot t) + 5 \text{ uA}$



0.1 Hz



3 hz



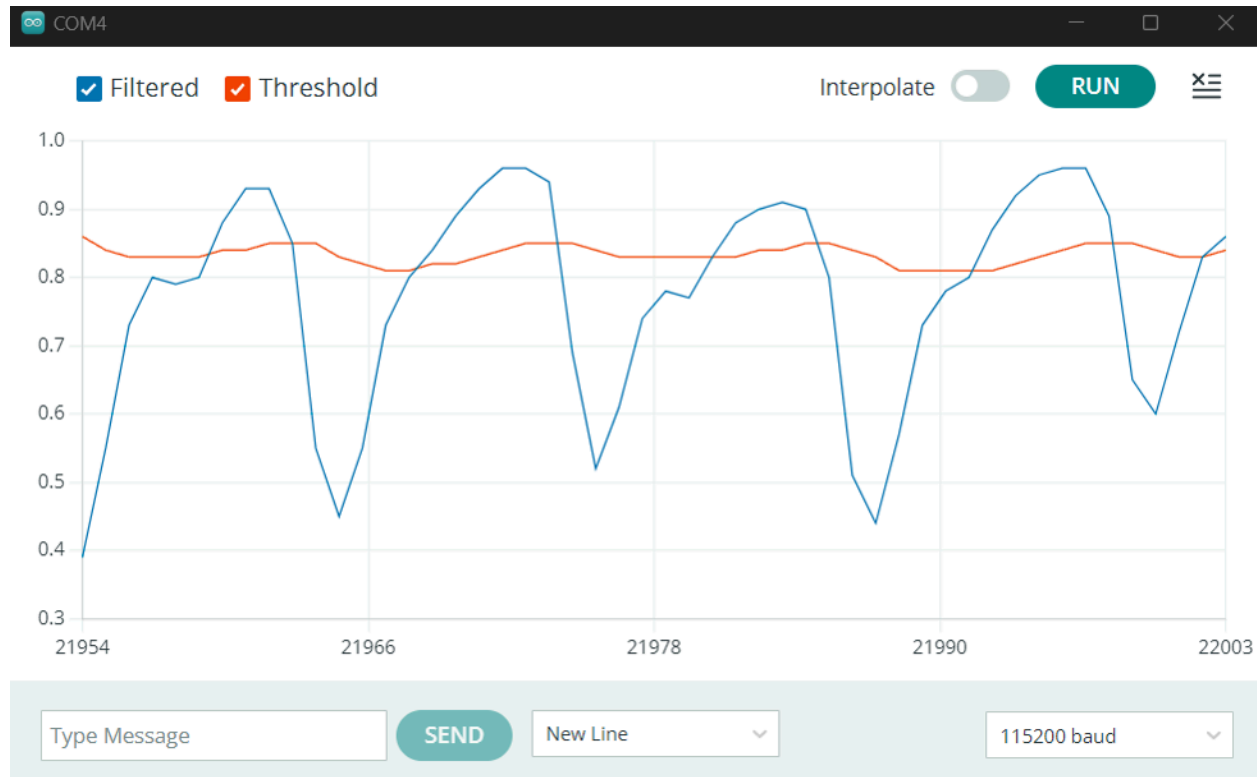
10 hz

Code

With the hardware finished, I moved on to the software side. I used Arduino IDE to write code for the ESP32 and use the serial monitor and plotter. I wanted the user of the device to easily see their BPM with their own eyes, so I added an LCD screen that displayed it and a red LED that blinked with each heartbeat.

Main ideas behind code:

- Voltage signal from circuit output is smoothed to make it less erratic, this signal is called “filtered”
- “Baseline” follows “filtered” at a slow rate, it’s not exactly the midpoint of the alternating filtered signal but it’s used as a baseline for the threshold
- “Threshold” is “baseline” + a constant
- A possible beat occurs when “filtered” > “threshold”
- Follows logic of a state machine, 3 states: WAITING, TRACKING_PEAK, REFRACTORY_STATE
- WAITING is the default state that waits until a possible beat occurs
- TRACKING_PEAK is the state that does most of the work, it decides whether the beat was valid or not. It does peak amplitude and peak interval checks and also sets the refractory period
- REFRACTORY_STATE is the state that the program lands after a beat was successfully detected. It’s like a cooldown period to prevent reading beats that happen too soon after.



Serial plotter showing Filtered and Threshold values over time

Shortcomings

The circuit is susceptible to motion, moving your finger too much might cause your BPM to shoot up, although it will return to normal values once the movement stops. This happens because the circuit can't differentiate between objects placed between the IR LED and photodiode, so although the code has measures against artifacts, even moving an object in and out of the sensor repeatedly might register as a heartbeat.